Módulo 11 - Aulas 1 e 2

Tarefas

No final deste módulo você deve submeter em um ÚNICO arquivo PDF os seguintes prints:

- Atividade 11.1: passo 6.
- Atividade 11.2: passo 8.
- Atividade 11.3: passo 8.
- Atividade 11.4: passo 8.
- Atividade 11.5: passo 14.

Regras para a elaboração do documento:

1. Antes de cada Print, adicione obrigatoriamente uma frase explicativa que sinalize do que se trata o print. A inserção de prints sem a devida frase explicativa será considerada como tentativa de atrapalhar a correção do instrutor e será penalizada a critério do instrutor. Exemplo:

Print da atividade 11.1: [Imagem com o Print]

2. Os Prints devem ser tirados da TELA CHEIA. Quando tirados da VM da AWS, devem ser capturados **obrigatoriamente** da tela cheia clicando no botão "Screenshot" → "Take screenshot" da barra de ferramentas do Hypervisor da AWS.
3. Insira **somente a quantidade de Prints solicitados por atividade** usando exclusivamente 1 página por print. **A página do documento onde você vai inserir o Print deve estar com a orientação no modo PAISAGEM** ´para termos melhor aproveitamento do espaço. Ou seja, seu documento deverá ter a mesma quantidade de páginas que a quantidade do total de Prints! A inserção de prints desnecessários será considerada como tentativa de atrapalhar a correção do instrutor e será penalizada com nota 0.

4. Apresente Prints legíveis e com tamanho correto para fácil leitura. O envio de prints com letras minúsculas poderá ser considerado como tentativa de atrapalhar a correção do instrutor e será penalizada a critério do instrutor.

Atividade 11.1 – Explorando a tabela ARP no Kali Linux

Nesta atividade, vamos usar o Kali Linux para explorar a tabela ARP e seu comportamento com conexões.

Vamos começar inicializando nosso Kali Linux via RDP ao IP: 192.168.98.40, com usuário “aluno” e senha “rnpesr”.

1. Abra o Terminal e execute o seguinte comando (com senha “rnpesr”) para ser super usuário:

```
┌──(aluno㉿kali)-[~]
└─$ sudo -i
[sudo] senha para aluno:
```

2. Agora vamos visualizar a tabela ARP atual do sistema com o seguinte comando:

```
┌──(root㉿kali)-[~]
└─# arp -a
ip-192-168-98-201.ec2.internal (192.168.98.201) em 12:5c:9d:d7:39:31
[ether] em eth0
ip-192-168-98-1.ec2.internal (192.168.98.1) em 12:82:f4:28:c9:8d
[ether] em eth0
ip-192-168-98-2.ec2.internal (192.168.98.2) em 12:82:f4:28:c9:8d
[ether] em eth0
```

A saída do comando possui os parâmetros:

- **Endereço IP (nome do host) (Endereço IP em notação decimal)**: O endereço IP da máquina, seguido opcionalmente pelo nome do host e pelo endereço IP em notação decimal.

- **em (Endereço MAC) [ether]**: "em" indica a interface de rede em que a máquina associada ao endereço IP está localizada, seguido pelo endereço MAC da máquina em notação hexadecimal e o tipo de protocolo de camada de enlace, neste caso, "ether" para Ethernet.
- **em (Nome da interface)**: Indica a interface de rede em que a máquina está localizada. Por exemplo, "em eth0" indica que a máquina está associada à interface de rede Ethernet 0.

3. Vamos limpa a Tabela ARP e visualizar a Tabela ARP vazia:

```
┌──(root㉿kali)-[~]
└─# ip -s -s neigh flush all
192.168.98.201 dev eth0 lladdr 12:5c:9d:d7:39:31  ref 1 used
66/0/66probes 1 REACHABLE
192.168.98.1 dev eth0 lladdr 12:82:f4:28:c9:8d  used 419/419/376probes
1 STALE
192.168.98.2 dev eth0 lladdr 12:82:f4:28:c9:8d  used 37/37/13probes 1
STALE

*** Round 1, deleting 3 entries ***
*** Flush is complete after 1 round ***
```

4. Veja novamente a Tabela ARP repetindo o passo 2:

```
┌──(root㉿kali)-[~]
└─# arp -a
ip-192-168-98-201.ec2.internal (192.168.98.201) em 12:5c:9d:d7:39:31
[ether] em eth0
ip-192-168-98-2.ec2.internal (192.168.98.2) em 12:82:f4:28:c9:8d
[ether] em eth0
```

5. Agora, abra o Mozilla Firefox (“Aplicativos” → “Navegador Web”) e entre no Google:

```
https://www.google.com
```

6. Repita o passo 4 e veja que agora volta a ser listada a rota via 192.168.98.1 (**NÃO SE ESQUEÇA DE PRINTAR ESTE PASSO!**):

```
┌──(root㉿kali)-[~]
└─# arp -a
ip-192-168-98-1.ec2.internal (192.168.98.1) em 12:82:f4:28:c9:8d
[ether] em eth0
ip-192-168-98-201.ec2.internal (192.168.98.201) em 12:5c:9d:d7:39:31
[ether] em eth0
ip-192-168-98-2.ec2.internal (192.168.98.2) em 12:82:f4:28:c9:8d
[ether] em eth0
```

7. Feche o Mozilla Firefox e o Terminal.

Parabéns! Agora você sabe identificar o conteúdo da Tabela ARP no seu Kali Linux!

Atividade 11.2 – Implementando Alias de Interface de Rede no Kali Linux

Nesta atividade, vamos usar o Kali Linux para criar e utilizar Interfaces Alias da Interface eth0 do Kali Linux.

Vamos começar inicializando nosso Kali Linux via RDP ao IP: 192.168.98.40, com usuário “aluno” e senha “rnpesr”.

1. Abra o Terminal e execute o seguinte comando (com senha “rnpesr”) para ser super usuário:

```
┌──(aluno㉿kali)-[~]
└─$ sudo -i
[sudo] senha para aluno:
```

2. Crie os Alias manualmente editando o arquivo:

```
┌──(root㉿kali)-[~]
└─# nano /etc/network/interfaces
```

3. Insira o seguinte texto no final do arquivo:

```
auto eth0:1
iface eth0:1 inet static
    address 192.168.98.41
    netmask 255.255.255.0

auto eth0:2
iface eth0:2 inet static
    address 192.168.98.42
    netmask 255.255.255.0
```

Essas linhas criam dois aliases para eth0 com os endereços IP 192.168.98.41 e 192.168.98.42, respectivamente. Feche e salve o arquivo apertando “Ctrl + x”, “s” e finalmente “ENTER”.

4. Reinicie a máquina para que sejam aplicadas as novas configurações de Aliases:

```
┌──(root㉿kali)-[~]
└─# reboot
```

5. Aguarde 3 minutos para inicializar o Kali Linux via RDP ao IP: 192.168.98.40, com usuário “aluno” e senha “rnpesr”.
6. Repita o passo 1 e verifique se os aliases foram configurados corretamente usando o seguinte comando:

```
┌──(aluno㉿kali)-[~]
└─$ sudo -i
[sudo] senha para aluno:

┌──(root㉿kali)-[~]
└─# ip addr show eth0
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9001 qdisc mq state UP
group default qlen 1000
    link/ether 12:4d:70:f0:ab:8b brd ff:ff:ff:ff:ff:ff
    inet 192.168.98.40/24 brd 192.168.98.255 scope global dynamic eth0
       valid_lft 3447sec preferred_lft 3447sec
    inet 192.168.98.41/24 brd 192.168.98.255 scope global secondary
eth0:1
       valid_lft forever preferred_lft forever
    inet 192.168.98.42/24 brd 192.168.98.255 scope global secondary
eth0:2
       valid_lft forever preferred_lft forever
    inet6 fe80::104d:70ff:fef0:ab8b/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
```

Veja os parâmetros da saída do comando anterior:

- **2: eth0**: Este é o nome da interface de rede.
- **<BROADCAST,MULTICAST,UP,LOWER_UP>**: Indica o status da interface. Neste caso, a interface está habilitada (UP) e pronta para transmitir e receber pacotes de broadcast e multicast.
- **mtu 9001**: Define o Maximum Transmission Unit (MTU) como 9001 bytes, indicando o uso de Jumbo Frames. Esse valor maior que o padrão (1500) melhora a eficiência em redes de alto desempenho, comuns em ambientes virtualizados (como AWS), onde a infraestrutura suporta quadros maiores.
- **qdisc mq state UP group default qlen 1000**: Detalhes sobre a disciplina de fila de transmissão (qdisc) e o comprimento máximo da fila (qlen) da interface.
- **link/ether 12:4d:70:f0:ab:8b**: Endereço MAC da interface.
- **inet 192.168.98.40/24 brd 192.168.98.255 scope global dynamic eth0**: Endereço IP principal da interface eth0, que foi atribuído dinamicamente via DHCP. O /24 indica a máscara de sub-rede. O brd 192.168.98.255 indica o endereço de broadcast da sub-rede.
- **inet 192.168.98.41/24 brd 192.168.98.255 scope global secondary eth0:1**: Um alias de endereço IP (eth0:1) associado à interface eth0.

- **inet 192.168.98.42/24 brd 192.168.98.255 scope global secondary eth0:2**: Outro alias de endereço IP (eth0:2) associado à interface eth0.
- **inet6 fe80::104d:70ff:fef0:ab8b/64 scope link**: Endereço IPv6 local da interface.

7. Após configurar os aliases da interface eth0 no Kali Linux, você pode usá-los da mesma forma que usa a interface principal eth0. Veja aseguir alguns exemplos de como você pode usar os aliases. Efetue o Ping para os IPs dos Aliases criados:

```
┌──(root㉿kali)-[~]
└─# ping 192.168.98.41
PING 192.168.98.41 (192.168.98.41) 56(84) bytes of data.
64 bytes from 192.168.98.41: icmp_seq=1 ttl=64 time=0.045 ms
64 bytes from 192.168.98.41: icmp_seq=2 ttl=64 time=0.044 ms
^C
--- 192.168.98.41 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1024ms
rtt min/avg/max/mdev = 0.044/0.044/0.045/0.000 ms

┌──(root㉿kali)-[~]
└─# ping 192.168.98.42
PING 192.168.98.42 (192.168.98.42) 56(84) bytes of data.
64 bytes from 192.168.98.42: icmp_seq=1 ttl=64 time=0.038 ms
64 bytes from 192.168.98.42: icmp_seq=2 ttl=64 time=0.044 ms
^C
--- 192.168.98.42 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1003ms
rtt min/avg/max/mdev = 0.038/0.041/0.044/0.003 ms
```

8. Estabeleça o acesso SSH para o IP 192.168.98.42 com usuário “aluno” e senha “rnpesr” (**NÃO SE ESQUEÇA DE PRINTAR ESTE PASSO!**):

```
┌──(root㉿kali)-[~]
└─# ssh aluno@192.168.98.42
The authenticity of host '192.168.98.42 (192.168.98.42)' can't be
established.
ED25519 key fingerprint is SHA256:hsrbT/3IfQ22bAUQPUE8T3dpJA/
aBVyt3wcyb6zKgO0.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])?
yes
Warning: Permanently added '192.168.98.42' (ED25519) to the list of
known hosts.
aluno@192.168.98.42's password:
Linux kali 6.6.9-cloud-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.6.9-1kali1
(2024-01-08) x86_64

The programs included with the Kali GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
┏(Message from Kali developers)
┃
┃ This is a minimal installation of Kali Linux, you likely
┃ want to install supplementary tools. Learn how:
┃ ⇒ https://www.kali.org/docs/troubleshooting/common-minimum-setup/
┃
┃ This is a cloud installation of Kali Linux. Learn more about
┃ the specificities of the various cloud images:
┃ ⇒ https://www.kali.org/docs/troubleshooting/common-cloud-setup/
┃
┗(Run: “touch ~/.hushlogin” to hide this message)
┌──(aluno㉿kali)-[~]
└─$ exit
sair
Connection to 192.168.98.42 closed.
```

9. Edite o seguinte arquivo e apague as linhas apresentadas no passo 3:

```
┌──(root㉿kali)-[~]
└─# nano /etc/network/interfaces
```

10. Feche e salve o arquivo apertando “Ctrl + x”, “s” e finalmente “ENTER”.

11. Reinicie a máquina para que sejam aplicadas as novas configurações sem Aliases:

```
┌──(root㉿kali)-[~]
└─# reboot
```

Parabéns! Agora você sabe como criar Aliases que podem ser usados na sua subrede!

Atividade 11.3 – Implementando o MAC Cloning no Kali Linux

Nesta atividade, vamos usar o Kali Linux para fazer a clonagem de endereço MAC na Interface docker0 que temos no Kali Linux.

Vamos começar inicializando nosso Kali Linux via RDP ao IP: 192.168.98.40, com usuário “aluno” e senha “rnpesr”.

1. Abra o Terminal e execute o seguinte comando (com senha “rnpesr”) para ser super usuário:

```
┌──(aluno㉿kali)-[~]
└─$ sudo -i
[sudo] senha para aluno:
```

2. Vamos conhecer os endereços MAC da nossa VM:

```
┌──(root㉿kali)-[~]
└─# ip link show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN
mode DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9001 qdisc mq state UP
mode DEFAULT group default qlen 1000
    link/ether 12:4d:70:f0:ab:8b brd ff:ff:ff:ff:ff:ff
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue
state DOWN mode DEFAULT group default
    link/ether 02:42:4c:36:59:3d brd ff:ff:ff:ff:ff:ff
```

A saída desse comando tem os seguintes endereços MAC:

- **lo (Loopback)**: 00:00:00:00:00:00
- **eth0**: 12:4d:70:f0:ab:8b
- **docker0**: 02:42:4c:36:59:3d

3. Outra forma de visualizar os endereços MAC com os respectivos IPs é:

```
┌──(root㉿kali)-[~]
└─# ifconfig
docker0: flags=4099<UP,BROADCAST,MULTICAST>  mtu 1500
        inet 172.17.0.1  netmask 255.255.0.0  broadcast 172.17.255.255
        ether 02:42:4c:36:59:3d  txqueuelen 0  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 6 overruns 0  carrier 0  collisions 0

eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 9001
        inet 192.168.98.40  netmask 255.255.255.0  broadcast
192.168.98.255
        inet6 fe80::104d:70ff:fef0:ab8b  prefixlen 64  scopeid
0x20<link>
        ether 12:4d:70:f0:ab:8b  txqueuelen 1000  (Ethernet)
        RX packets 2674  bytes 160101 (156.3 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 4507  bytes 22743043 (21.6 MiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Loopback Local)
        RX packets 15  bytes 1157 (1.1 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 15  bytes 1157 (1.1 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

4. Veja que o endereço MAC do docker0 é “02:42:4c:36:59:3d” neste caso. **IMPORTANTE: No seu caso esse endereço MAC provavelmente será diferente, use o endereço MAC apresentado na sua VM!**. Agora, vamos substituir o endereço do docker0 por “08:00:27:8b:c0:05”. Antes de alterar o endereço MAC, é recomendável desativar a interface de rede docker0:

```
┌──(root㉿kali)-[~]
└─# ip link set docker0 down
```

5. Agora, usaremos o comando "ip" para clonar o endereço MAC do docker0 pelo endereço “08:00:27:8b:c0:05”:

```
┌──(root㉿kali)-[~]
└─# ip link set dev docker0 address 08:00:27:8b:c0:05
```

6. Após clonar o endereço MAC, reativaremos a interface de rede docker0:

```
┌──(root㉿kali)-[~]
└─# ip link set docker0 up
```

7. Agora, vamos ver se funcionou listando as interfaces:

```
┌──(root㉿kali)-[~]
└─# ip link show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN
mode DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9001 qdisc mq state UP
mode DEFAULT group default qlen 1000
    link/ether 12:4d:70:f0:ab:8b brd ff:ff:ff:ff:ff:ff
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue
state DOWN mode DEFAULT group default
    link/ether 08:00:27:8b:c0:05 brd ff:ff:ff:ff:ff:ff
```

Veja que a interface de rede docker0 está com o endereço MAC “08:00:27:8b:c0:05”.

8. Agora, vamos reverter o processo atribuindo a docker0 o endereço MAC original dele “02:42:4c:36:59:3d” ou outro conforme endereço apresentado na sua VM e listado nos passos 2 e 3 (**NÃO SE ESQUEÇA DE PRINTAR ESTE PASSO!**):

```
┌──(root㉿kali)-[~]
└─# ip link set docker0 down

┌──(root㉿kali)-[~]
└─# ip link set dev docker0 address 02:42:4c:36:59:3d

┌──(root㉿kali)-[~]
└─# ip link set docker0 up
```

9. Verifique que a interface de rede docker0 voltou a ter o endereço MAC original:

```
┌──(root㉿kali)-[~]
└─# ip link show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN
mode DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9001 qdisc mq state UP
mode DEFAULT group default qlen 1000
    link/ether 12:4d:70:f0:ab:8b brd ff:ff:ff:ff:ff:ff
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue
state DOWN mode DEFAULT group default
    link/ether 02:42:4c:36:59:3d brd ff:ff:ff:ff:ff:ff
```

10. Feche o Terminal.

Parabéns! Agora você saber implementar o MAC Cloning e clonar ou substituir endereços MAC das placas de rede do seu computador. Lembre-se de que o MAC Cloning pode ter implicações legais e de segurança, dependendo do contexto em que é usado. Certifique-se de estar ciente das políticas de rede e leis locais antes de realizar o MAC Cloning, e use-o apenas com propósitos legítimos e éticos. Se usado para cometer crimes cibernéticos, você estará infringindo a Lei Carolina Dieckmann.

Atividade 11.4 – Configurando QoS no Kali Linux

Nesta atividade, vamos usar o Kali Linux definir parâmetros de Qualidade de Serviço (QoS) nas placas de rede do Kali Linux.

Vamos começar inicializando nosso Kali Linux via RDP ao IP: 192.168.98.40, com usuário “aluno” e senha “rnpesr”.

1. Abra o Terminal e execute o seguinte comando (com senha “rnpesr”) para ser super usuário:

```
┌──(aluno㉿kali)-[~]
└─$ sudo -i
[sudo] senha para aluno:
```

2. Veja as interfaces de rede presentes e seus IPs:

```
┌──(root㉿kali)-[~]
└─# ifconfig
docker0: flags=4099<UP,BROADCAST,MULTICAST>  mtu 1500
        inet 172.17.0.1  netmask 255.255.0.0  broadcast 172.17.255.255
        ether 02:42:4c:36:59:3d  txqueuelen 0  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 14 overruns 0  carrier 0  collisions 0

eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 9001
        inet 192.168.98.40  netmask 255.255.255.0  broadcast
192.168.98.255
        inet6 fe80::104d:70ff:fef0:ab8b  prefixlen 64  scopeid
0x20<link>
        ether 12:4d:70:f0:ab:8b  txqueuelen 1000  (Ethernet)
        RX packets 6556  bytes 418883 (409.0 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 9742  bytes 45663359 (43.5 MiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Loopback Local)
        RX packets 15  bytes 1157 (1.1 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 15  bytes 1157 (1.1 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

3. Em seguida, vamos priorizar o tráfego ICMP (ping) em relação a outros tráfegos na placa “eth0”:

```
┌──(root㉿kali)-[~]
└─# tc qdisc add dev eth0 root handle 1: prio

┌──(root㉿kali)-[~]
└─# tc filter add dev eth0 parent 1: protocol ip prio 1 u32 match ip
protocol 1 0xff flowid 1:1
```

Essa configuração é feita através do uso de filas de disciplina (qdisc) e filtros (filter). Veja os parâmetros do primeiro comando:

- **tc**: O comando utilizado para configurar o controle de tráfego (Traffic Control) no Linux.
- **qdisc**: É a abreviação de "queueing discipline", que se refere à disciplina de filas aplicada a uma interface de rede para controlar o tráfego.
- **add**: Indica que estamos adicionando uma nova disciplina de filas.
- **dev eth0**: Especifica a interface de rede à qual a disciplina de filas será aplicada, no caso, a interface eth0.
- **root**: Indica que a disciplina de filas será aplicada à raiz da hierarquia de filas da interface, ou seja, será aplicada diretamente na interface.
- **handle 1: prio**: O "handle" é um identificador único para a disciplina de filas. Neste caso, estamos utilizando o handle "1:" e atribuindo a disciplina de filas do tipo "prio" (Priority Queueing).

Através desse comando, estamos criando uma disciplina de filas do tipo "prio" (Priority Queueing) na interface eth0, que permitirá priorizar o tráfego com base em níveis de prioridade.

Os parâmetros do segundo comando são:

- **tc**: O comando utilizado para configurar o controle de tráfego (Traffic Control) no Linux.
- **filter**: É utilizado para adicionar um filtro para classificar os pacotes com base em suas características.
- **add**: Indica que estamos adicionando um novo filtro.

- **dev eth0**: Especifica a interface de rede na qual o filtro será aplicado, no caso, a interface eth0.
- **parent 1:**: Especifica o handle da disciplina de filas à qual o filtro será aplicado. Neste caso, estamos aplicando o filtro à disciplina de filas criada anteriormente com o handle "1:".
- **protocol ip**: Especifica que estamos filtrando pacotes IP (camada de rede).
- **prio 1**: Define a prioridade do filtro, que neste caso é 1.
- **u32**: Indica que usaremos uma correspondência u32 (userspace 32) para definir as condições do filtro.
- **match ip protocol 1 0xff**: Define a correspondência u32 para filtrar pacotes com o protocolo IP igual a 1 (ICMP). O "0xff" é uma máscara que corresponde ao valor exato do protocolo.
- **flowid 1:1**: Define o identificador do fluxo para onde os pacotes que atendem às condições do filtro serão direcionados. Neste caso, os pacotes serão direcionados para o handle "1:".

Os 2 comandos criam uma política de priorização para pacotes ICMP (ping) na interface eth0, dando a eles prioridade de nível 1. Isso significa que os pacotes ICMP serão tratados com prioridade mais alta do que outros pacotes na fila, melhorando o tempo de resposta para esses pacotes e garantindo um tratamento preferencial para o tráfego de ping.

4. O Linux permite apenas uma QDisc raiz por interface de rede e ela já está em uso. Remova a QDisc existente:

```
┌──(root㉿kali)-[~]
└─# tc qdisc del dev eth0 root
```

5. Veja que há conectividade com o IP 192.168.98.1:

```
┌──(root㉿kali)-[~]
└─# ping 192.168.98.1
PING 192.168.98.1 (192.168.98.1) 56(84) bytes of data.
64 bytes from 192.168.98.1: icmp_seq=1 ttl=255 time=0.325 ms
64 bytes from 192.168.98.1: icmp_seq=2 ttl=255 time=0.424 ms
64 bytes from 192.168.98.1: icmp_seq=3 ttl=255 time=0.450 ms
^C
--- 192.168.98.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2040ms
rtt min/avg/max/mdev = 0.325/0.399/0.450/0.053 ms
```

6. Agora, vamos priorizar pacotes vindo do IP 192.168.98.1 com fila hierárquica:

```
┌──(root㉿kali)-[~]
└─# tc qdisc add dev eth0 root handle 1: htb

┌──(root㉿kali)-[~]
└─# tc class add dev eth0 parent 1: classid 1:1 htb rate 10mbit

┌──(root㉿kali)-[~]
└─# tc filter add dev eth0 parent 1: protocol ip prio 1 u32 match ip
src 192.168.98.1 flowid 1:1
```

Os comandos são usados para configurar o controle de tráfego no Linux usando o tc (Traffic Control). Eles são usados para criar uma disciplina de filas hierárquica (HTB - Hierarchical Token Bucket) na interface de rede eth0 e aplicar uma política de limitação de taxa para um determinado endereço IP (192.168.98.1). Vamos explicar o primeiro comando:

- **tc**: O comando utilizado para configurar o controle de tráfego (Traffic Control) no Linux.
- **qdisc**: É a abreviação de "queueing discipline", que se refere à disciplina de filas aplicada a uma interface de rede para controlar o tráfego.
- **add**: Indica que estamos adicionando uma nova disciplina de filas.
- **dev eth0**: Especifica a interface de rede à qual a disciplina de filas será aplicada, no caso, a interface eth0.
- **root**: Indica que a disciplina de filas será aplicada à raiz da hierarquia de filas da interface, ou seja, será aplicada diretamente na interface.

- **handle 1:**: O "handle" é um identificador único para a disciplina de filas. Neste caso, estamos utilizando o handle "1:".
- **htb**: É o tipo de disciplina de filas que estamos adicionando, neste caso, o Hierarchical Token Bucket (HTB). O HTB permite criar uma hierarquia de filas para controle de tráfego mais complexo.

O segundo comando consiste em:

- **tc**: O comando utilizado para configurar o controle de tráfego (Traffic Control) no Linux.
- **class**: É utilizado para adicionar uma classe de tráfego, que é uma das filas na hierarquia HTB.
- **add**: Indica que estamos adicionando uma nova classe de tráfego.
- **dev eth0**: Especifica a interface de rede na qual a classe de tráfego será aplicada, no caso, a interface eth0.
- **parent 1:**: Especifica o handle da disciplina de filas (HTB) à qual a classe de tráfego pertencerá. Neste caso, estamos aplicando a classe à disciplina de filas criada anteriormente com o handle "1:".
- **classid 1:1**: Define o identificador único da classe de tráfego. Neste caso, estamos utilizando o classid "1:1".
- **htb rate 10mbit**: Define a taxa de transmissão máxima permitida para essa classe de tráfego em 10 megabits por segundo (10mbit). Isso limitará a largura de banda disponível para essa classe de tráfego a 10 Mbps.

O terceiro comando consiste em:

- **tc**: O comando utilizado para configurar o controle de tráfego (Traffic Control) no Linux.
- **filter**: É utilizado para adicionar um filtro para classificar os pacotes com base em suas características.
- **add**: Indica que estamos adicionando um novo filtro.
- **dev eth0**: Especifica a interface de rede na qual o filtro será aplicado, no caso, a interface eth0.
- **parent 1:**: Especifica o handle da disciplina de filas (HTB) à qual o filtro será aplicado. Neste caso, estamos aplicando o filtro à disciplina de filas criada anteriormente com o handle "1:".

- **protocol ip**: Especifica que estamos filtrando pacotes IP (camada de rede).
- **prio 1**: Define a prioridade do filtro, que neste caso é 1.
- **u32**: Indica que usaremos uma correspondência u32 (userspace 32) para definir as condições do filtro.
- **match ip src 192.168.98.1: Define a correspondência u32 para filtrar pacotes com o endereço IP de origem igual a 192.168.98.1.
- **flowid 1:1**: Define o identificador do fluxo para onde os pacotes que atendem às condições do filtro serão direcionados. Neste caso, os pacotes serão direcionados para o classid "1:1" criado anteriormente.

7. Em seguida, vamos limitar a largura de banda de uma interface de rede eth0 para 1 Mbps. Antes disso, precisamos remover a disciplina de filas existente na interface eth0 antes de adicionar uma nova. Para remover a disciplina de filas existente, **execute o passo 4**.
8. Agora sim, vamos limitar a largura de banda de uma interface de rede eth0 para 1 Mbps (**NÃO SE ESQUEÇA DE PRINTAR ESTE PASSO!**):

```
┌──(root㉿kali)-[~]
└─# tc qdisc add dev eth0 root tbf rate 1mbit burst 10kb latency 50ms
```

Veja que a conexão fica lenta! Os detalhes do comando anterior:

- **tc**: O comando utilizado para configurar o controle de tráfego (Traffic Control) no Linux.
- **qdisc**: É a abreviação de "queueing discipline", que se refere à disciplina de filas aplicada a uma interface de rede para controlar o tráfego.
- **add**: Indica que estamos adicionando uma nova disciplina de filas.
- **dev eth0**: Especifica a interface de rede à qual a disciplina de filas será aplicada, no caso, a interface eth0.
- **root**: Indica que a disciplina de filas será aplicada à raiz da hierarquia de filas da interface, ou seja, será aplicada diretamente na interface.
- **tbf**: É o tipo de disciplina de filas que estamos adicionando, neste caso, o Token Bucket Filter (TBF).

- **rate 1mbit**: Define a taxa de transmissão máxima permitida para essa interface em 1 megabit por segundo (1mbit). Isso limitará a largura de banda disponível para a interface a 1 Mbps.
- **burst 10kb**: Especifica o tamanho máximo do burst, que é a quantidade máxima de dados que pode ser transmitida em uma rajada. Neste caso, o tamanho máximo do burst é de 10 kilobytes (10kb).
- **latency 50ms**: Define a latência ou atraso da fila em 50 milissegundos (50ms). Isso afeta a rapidez com que o tráfego é enviado pela interface após ser colocado na fila.

9. **Execute o passo 4 para remover a limitação e voltar à taxa de transmissão normal**. Feche o Terminal.

Parabéns! Agora você sabe como configurar os parâmetros básicos de QoS no Linux!

Atividade 11.5 – Configurando QoS no Windows Server 2022

Nesta Atividade configuraremos parâmetros das conexões de rede em função de QoS.

1. Vamos começar inicializando o Windows Server 2022 (cliente) via RDP ao IP: 192.168.98.30. Certifique-se que o nome de usuário é "WINSERVER\Administrator" (não "ALUNO\nome1"), com usuário "Administrator" (não "nome1") e senha "RnpEsr123@" (não "S3nh@nom31").
2. Abra o Microsoft Edge e meça a taxa de transferência:

```
https://www.minhaconexao.com.br/
```

3. Veja que o Upload resultou acima de 20Mbps! Feche o Microsoft Edge.
4. Clique onde está escrito "Type here to search", escreva "gpedit.msc" e clique no botão "gpedit.msc" para abrir o gpedit.msc.

5. Na nova janela “Local Group Policy Editor”, expanda “Computer Configuration” → “Windows Settings”.
6. Clique com o botão direito em “Policy-based QoS” e clique em “Create new policy ...”.
7. Na janela “Policy-based QoS”, insira o nome “Teste” no campo “Policy name”.
8. O DSCP é um campo de 6 bits no cabeçalho IP que permite a marcação de pacotes para indicar sua prioridade de tratamento dentro da rede (número entre 0 e 63). Ao especificar um valor DSCP em uma política de QoS, você está designando um nível de prioridade para o tráfego correspondente. Deixe o valor em 0.
9. Specify Outbound Throttle Rate permite limitar a velocidade de transmissão de pacotes de dados de saída para garantir um uso equilibrado e eficiente da largura de banda da rede. Ative o campo “Specify Outbound Throttle Rate” e configure seu valor para 1 MBps e clique em “Next”
10. Deixe selecionado em “All applications” e clique em “Next”.
11. Deixe selecionados os 2 campos em “Any source IP address” e clique em “Next”.
12. Deixe selecionados os campos “From any source port” e “From any destination port” e clique em “Finish”. Agora, todas suas conexões ficarão limitadas a 1 MBps ou o equevialente de 8 Mbps.
13. Veja que no campo direito da janela “Local Group Policy Editor”, a regra “Teste” foi criada.
14. Repita o passo 2 e veja agora que a taxa de upload ficou entre 7Mbps a 8Mbps (**NÃO SE ESQUEÇA DE PRINTAR ESTE PASSO!**).
15. Volte a janela do item 12 e apague a regra com o botão direito na regra → “Delete policy” → “Yes”. Feche todas as janelas.

Parabéns! Agora você sabe aplicar o QoS na limitação de Upload no Windows Server 2022!